Online Traffic Fine Payment System

(OLMS – Jurmana)

Bachelor of Technology
Computer Science and Engineering

Submitted By

Ragyam Sinha (13000123064)

September 2025

Techno Main
EM-4/1, Sector-V, Salt Lake
Kolkata- 700091
West Bengal
India

Contents

1. Introduction 3

2. Life–Cycle Model Used and its Justification 3

3. Design 3

i) DFD – 0 (Context Diagram) 4

ii) DFD – 1 (Detailed Diagram) 5

iii) DFD – 2 (Application & Approval Processes) 6

iv) Complete Data Dictionary 8

4. Size Estimation using FP Metric 10

a) Estimate the size of each module 10

5. COCOMO Estimations 10

a) Justify the choice of COCOMO Model 10

b) Justify the choice of Product Class for each module 11

c) Effort 11

d) Development Time 11

e) Development Cost 11

f) Staff Requirement 11

6. Conclusion 11

7. References 11

1. Introduction

The Online Traffic Fine Payment System (OTFPS – JURMANA) is a web-based application designed to digitize the process of issuing, managing, and paying traffic fines. It allows **citizens** to view and pay fines online, **admins (traffic police/officials)** to issue fines and manage disputes, and integrates with **external systems** like Parivahan DB, payment gateways, and SMS/Email providers.

The system aims to reduce manual errors, improve transparency, and provide 24/7 accessibility for citizens while supporting government compliance with traffic rules and digital governance initiatives.

2. Life–Cycle Model Used and its Justification

For the development of LeaveSync, the **Iterative Development Model** has been selected. This model breaks down the development of a large application into smaller, more manageable parts or iterations. Each iteration goes through its own cycle of requirements, design, implementation, and testing, resulting in a functional piece of the software.

Justification:

The **Iterative Waterfall Model** (or Incremental Development Model) is most appropriate for this project.

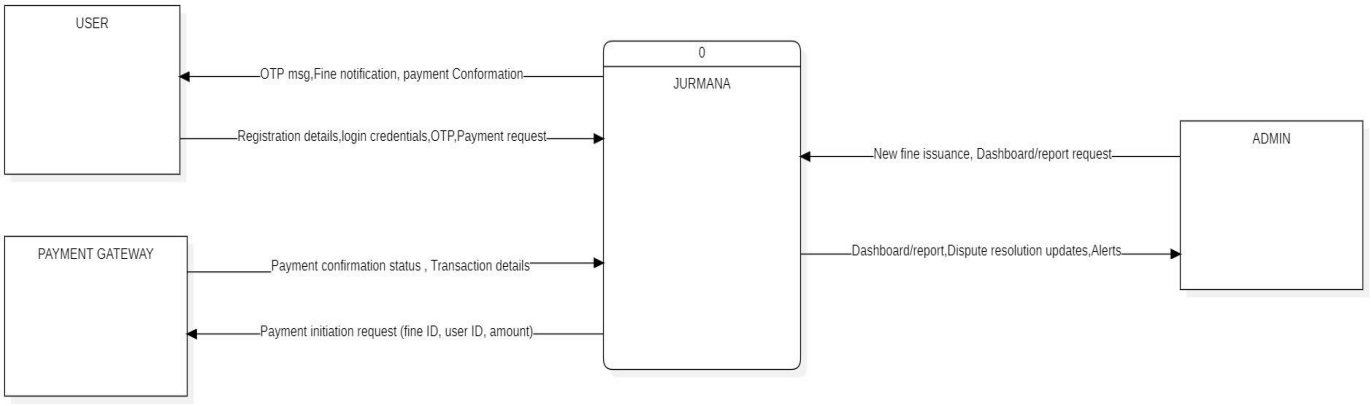
- **Why?**
    - The system requirements are well-defined (registration, fine management, payment, disputes, notifications, reports, EV rules).
    - Modules can be built and tested incrementally (e.g., start with user registration → fine issuance → payment → disputes).
    - Supports integration with external APIs (Parivahan, Payment Gateway, SMS/Email), which can be developed in later iterations.
    - Reduces risks compared to a pure waterfall model by allowing feedback at each stage while still maintaining a structured development cycle.

Thus, the **Iterative Waterfall / Incremental Model** ensures a balance between structured planning and flexibility.

3. Design

The design of the system is modelled using Data Flow Diagrams (DFDs) to visualize the flow of information. The comprehensive data dictionary below provides a detailed repository of all data elements, processes, and stores used in the system, decomposed to multiple levels.

i) DFD – 0 (Context Diagram)



Overview: he Level 0 DFD, or **Context Diagram**, represents the **Online Traffic Fine Payment System (OTFPS – JURMANA)** as a single high-level process. It defines the boundaries of the system by showing how it interacts with external entities. This diagram highlights the **key inputs received** by the system and the **outputs it generates**, without exposing any internal processing complexity.

Process:
0 OTFPS – JURMANA: This central process handles all the major functionalities including user registration/authentication, fine management, online payment, notifications, dispute handling, and reporting.

External Entities:

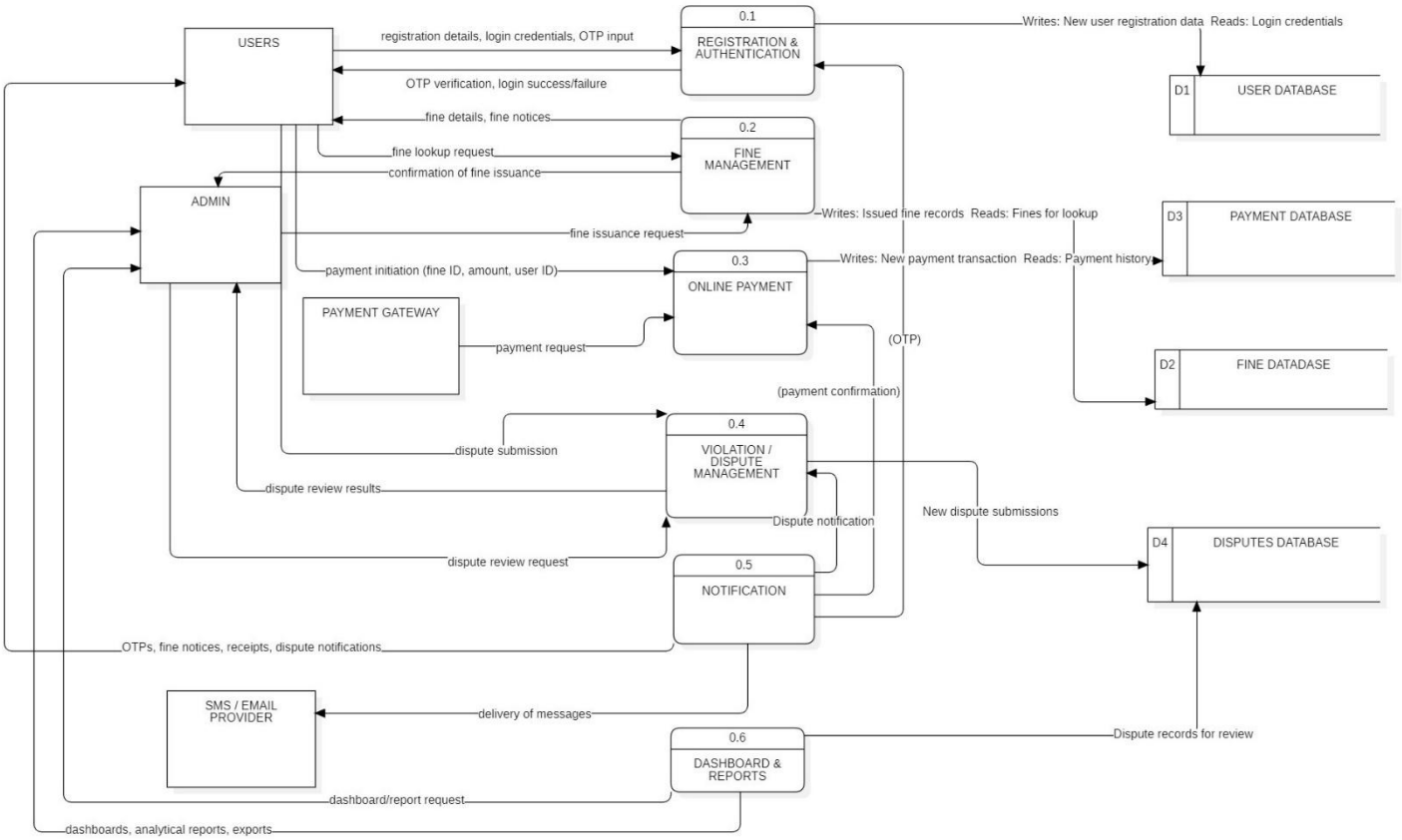
The system interacts with **three main external entities**:

1. **User (Citizen/Vehicle Owner)**
   - Provides: registration details, login credentials, OTP, payment requests.
   - Receives: OTP messages, fine notifications, payment confirmations.
2. **Admin (Traffic Authority/Official)**
   - Provides: new fine issuance data, dashboard/report requests.
   - Receives: dashboards, reports, dispute resolution updates, alerts.
3. **Payment Gateway**
   - Provides: payment confirmation status, transaction details.
   - Receives: payment initiation requests (fine ID, user ID, amount).

Data Stores:

At this level (DFD-0 / Context Diagram), **no internal data stores are shown**, since the focus is only on **external interactions** between the system and its entities.

ii) DFD – 1 (Detailed Diagram)



Overview:

The Level 1 DFD expands on the Level 0 Context Diagram by breaking the single **OTFPS – JURMANA** process into its **major sub-systems**. It provides a detailed view of the key functional modules, shows how each module interacts with external entities, and introduces the internal databases (data stores) where persistent information is kept. This level highlights the flow of data between **users, admins, payment gateways, notification services**, and the internal processes of the system.

Processes:

- **0.1 Registration & Authentication**
  Handles user/admin registration, login, and OTP verification.
  ↔ Reads/writes **User Database (D1)**
- **0.2 Fine Management**
  Enables admins to issue fines and users to view fine details.
  ↔ Reads/writes **Fine Database (D2)**

- **0.3 Online Payment**
  Processes payment initiation, forwards to payment gateway, and updates records.
  ↔ Reads/writes **Payment Database (D3)**
- **0.4 Violation/Dispute Management**
  Allows users to submit disputes and admins to review them.
  ↔ Reads/writes **Disputes Database (D4)**
- **0.5 Notification System**
  Sends OTPs, fine notices, receipts, and dispute updates via SMS/Email provider.
- **0.6 Dashboards & Reports**
  Generates analytical dashboards and custom reports for admins.
- users and generating business reports for management.

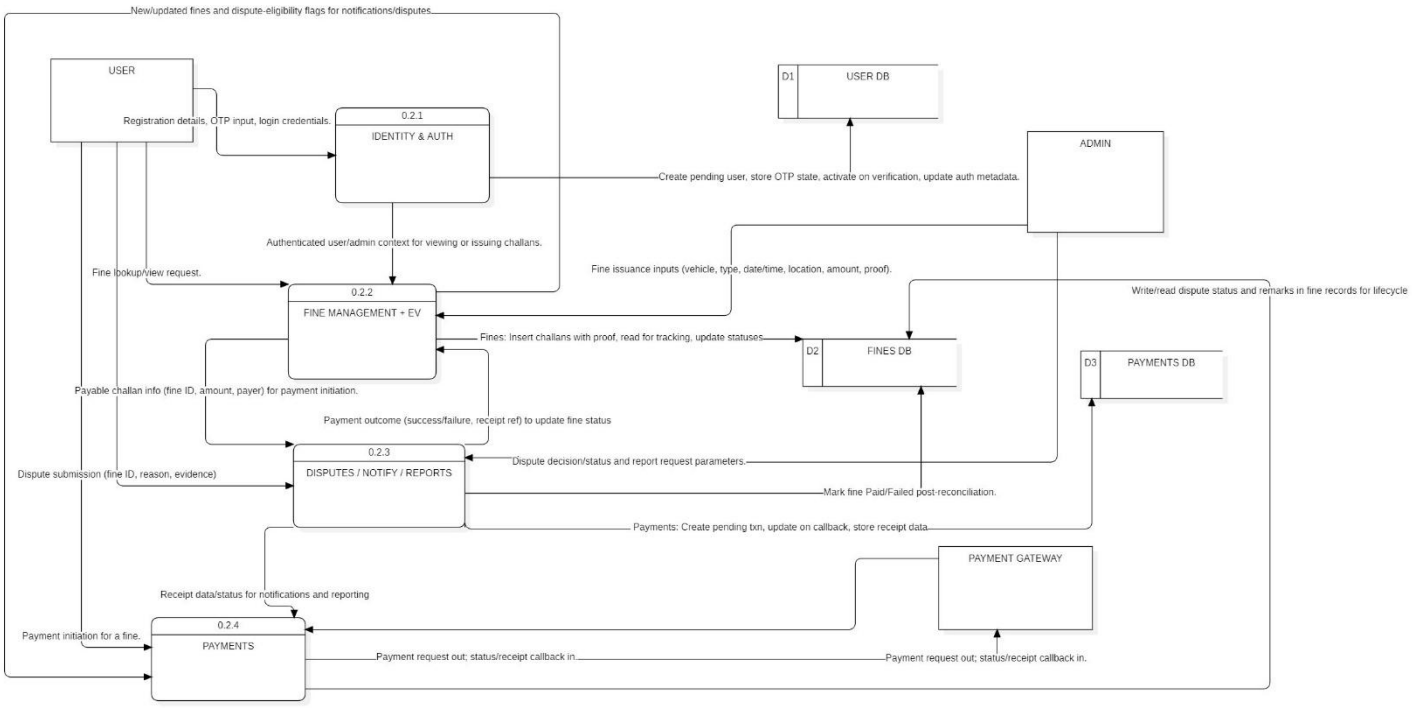
External Entities:

- **Users (Citizens/Vehicle Owners)**: Provide registration details, login credentials, OTP, fine lookup requests, payments, and disputes. Receive OTPs, fine notices, receipts, and dispute updates.
- **Admin (Traffic Authorities/Officials)**: Provide fine issuance details, dashboard/report requests, and dispute review decisions. Receive confirmation of fine issuance, analytical dashboards, and alerts.
- **Payment Gateway**: Processes fine payment requests and returns transaction confirmations.
- **SMS/Email Provider**: Delivers OTPs, fine notices, and confirmation messages to users.

Data Stores:

- **D1 User Database** – Stores user and admin credentials, profiles, and authentication data.
- **D2 Fine Database** – Maintains fine records (issued, paid, disputed).
- **D3 Payment Database** – Stores all payment transaction records.
- **D4 Disputes Database** – Contains dispute submissions and resolution history.

iii) DFD – 2 (Application & Approval Processes)



Overview:

This Level 2 DFD provides a more detailed view by expanding the **0.2 Fine Management + EV** process from the Level 1 diagram. It breaks down how fines are created, validated, stored, and updated, including integration with the payment and dispute modules. The diagram also shows how fine records interact with internal data stores and external entities.

Processes:

- **0.2.1 Identity & Authentication**
  Validates the user or admin credentials before allowing access to fine-related operations.
  ↔ Interacts with User Database (D1) for login/verification.
- **0.2.2 Fine Management + EV**
  Handles issuance of new fines by the admin, tracks fine details, and incorporates EV-specific rules (e.g., exemptions or special handling).

↔ Reads/writes Fine Database (D2).

- **0.2.3 Disputes / Notification / Reports**
  Manages user disputes on fines, generates reports, and prepares notifications (fine status, dispute outcomes).
  ↔ Reads/writes Disputes Database (D4).
- **0.2.4 Payments**
  Initiates fine payments, communicates with the payment gateway, and updates payment outcomes (success/failure).
  ↔ Reads/writes Payment Database (D3).

External Entities:

- **User**: Provides registration details, OTP input, login credentials, fine lookup requests, disputes, and payment initiation. Receives OTPs, fine notices, receipts, and dispute decisions.
- **Admin**: Provides fine issuance inputs (vehicle no., violation type, proof, etc.), requests reports, and reviews disputes. Receives confirmations, dashboards, and alerts.
- **Payment Gateway**: Processes fine payment requests and sends back confirmation/transaction results.

Data Stores:

- **D1 User Database** – Stores registration/login data and OTP verification states.
- **D2 Fine Database** – Stores all issued fines, tracking details, and EV eligibility flags.
- **D3 Payment Database** – Maintains payment transactions and receipts linked to fines.
- **D4 Disputes Database** – Stores dispute submissions, admin reviews, and resolution history.

iv) Complete Data Dictionary

Name	Type	Description	Composition	Source	Destination	Format
Registration Details	Data Flow	Info required for user account creation	Name, Mobile, Email, Vehicle No., Password	User	0.1 Registration & Authentication	Text/Alphanumeric
Login Credentials	Data Flow	Credentials for authentication	User ID, Password	User/Admin	0.1 Registration & Authentication	Text (Encrypted)
OTP Input	Data Flow	OTP entered by user for verification	OTP Code (6 digits)	User	0.1 Registration & Authentication	Numeric (6-digit)
OTP Verification Result	Data Flow	Response after OTP check	Status (Success/Failure), Message	0.1 Registration & Authentication	User	Text Message
User Data	Data Store (D1)	Stores registered user/admin details	User ID, Name, Contact, Vehicle No., Password, OTP state	0.1 Registration & Authentication	All Auth Processes	Relational DB
Fine Issuance Request	Data Flow	Admin request to create fine	Vehicle No., Violation, Date/Time, Location, Amount, Proof	Admin	0.2 Fine Management	Structured Record
Fine Details / Notices	Data Flow	User fines issued or retrieved	Fine ID, Amount, Status, Proof, Due Date	0.2 Fine Management	User	Structured Record
Fine Lookup Request	Data Flow	Query for fine records	User ID / Vehicle No.	User	0.2 Fine Management	Text / Numeric
Fine Records	Data Store (D2)	Stores fine details and status	Fine ID, Vehicle No., Violation, Amount, Status, Proof	0.2 Fine Management	0.2, 0.4, 0.6	Relational DB
Payment Initiation Request	Data Flow	User request to pay fine	Fine ID, User ID, Amount, Method	User	0.3 Online Payment	Structured Request
Payment Request	Data Flow	Payment forwarded to gateway	Fine ID, User ID, Amount, Method	0.3 Online Payment	Payment Gateway	API JSON/XML
Payment Confirmation	Data Flow	Result from payment gateway	Transaction ID, Status, Timestamp, Amount	Payment Gateway	0.3 Online Payment	API JSON/XML

Name	Type	Description	Composition	Source	Destination	Format
Payment Records	Data Store (D3)	Stores all transactions	Payment ID, Fine ID, User ID, Amount, Date, Status	0.3 Online Payment	0.3, 0.6	Relational DB
Receipt / Confirmation	Data Flow	Confirmation/receipt to user	Transaction ID, Fine ID, Amount, Status	0.3 Online Payment	User	PDF/Email/SMS
Dispute Submission	Data Flow	User raises complaint on a fine	Fine ID, Reason, Evidence	User	0.4 Dispute Management	Text + File (Image/PDF)
Dispute Review Request	Data Flow	Admin reviews disputes	Dispute ID, Comments, Status Update	Admin	0.4 Dispute Management	Text
Dispute Updates	Data Flow	Results of dispute decisions to users	Dispute ID, Status, Remarks	0.4 Dispute Management	User	Text / Email
Dispute Records	Data Store (D4)	Stores user disputes and admin resolutions	Dispute ID, Fine ID, Reason, Evidence, Status, Admin Remarks	0.4 Dispute Management	0.4, 0.6	Relational DB
Notifications	Data Flow	Alerts and messages to users	OTP, Fine Notice, Payment Receipt, Dispute Update	0.5 Notification	SMS/Email Provider	SMS/Email
Delivery of Messages	Data Flow	Sent via external provider	OTPs, Alerts, Confirmations	SMS/Email Provider	Users	SMS/Email
Dashboards/Reports	Data Flow	Summaries for admin	Reports of fines, disputes, payments	0.6 Dashboards & Reports	Admin	PDF/Excel/Graphical UI

4. Size Estimation using FP Metric

Function Point (FP) analysis is a standardized method used to measure the size of a software system based on its functionality from a user's perspective. It is independent of the programming language or technology used, making it a reliable metric for estimation.

a) Estimate the size of each module

The analysis for the **JURMANA system** was conducted by identifying key user functions from the Data Flow Diagrams (DFDs) and categorizing them into four logical modules. The final adjusted Function Point (FP) size for each module is summarized below.

Module Name	Unadjusted FP (UFP)	Adjusted FP
1. User & Authentication Module	22	23.1
2. Fine & Payment Management	44	46.2
3. Dispute Management Module	16	16.8
4. Notification & Reporting	18	18.9
Total Project Size	**100**	**105**

The detailed, step-by-step calculations for this Function Point analysis are available in the following **spreadsheet**

5. COCOMO Estimations

The Constructive Cost Model (COCOMO) is used to provide estimations for the project's effort, schedule, and staffing requirements based on the estimated size of the software.

spreadsheet

Below is a summary of the methodology and final results derived from those calculations.

Justify the choice of COCOMO Model

The Basic COCOMO model is chosen because this is an early-stage estimation for a medium-scale web application. It provides a quick and straightforward estimate based on the software's estimated size.

b) Justify the choice of Product Class

The Semi-Detached product class is selected for this project.

- Justification: JURMANA integrates multiple external systems (Payment Gateway, SMS/Email Provider, Parivahan DB), involves both users and admins, and requires more coordination than a simple organic system. The complexity is moderate but not as high as embedded real-time systems, making Semi-Detached the best fit.

- **Inputs / conversions**
  - Adjusted FP = **105**
  - LOC/FP = **60** (assumption)
  - **Total LOC** = 105 × 60 = **6,300 LOC**
  - **KLOC** = 6,300 / 1,000 = **6.3 KLOC**

a) Effort

- **Total LOC** = 105 FP × 60 LOC/FP = **6,300 LOC = 6.3 KLOC**
- **Effort (E)** = 3.0 × (6.3)^1.12 ≈ **23.57 Person-Months**

b) Development Time

- **Time (D) = 0.35 × (23.57) ≈ 8.25 Months**

c) Development Cost

- **Assuming an average loaded cost of ₹50,000 per person per month**
- **Cost** = 23.57 × 50,000 ≈ **₹11,78,500**
- **Average Cost per Month** = ₹11,78,500 ÷ 8.25 ≈ **₹1,42,800 / month**

d) Staff Requirement

- **Staff (S)** = 23.57 ÷ 8.25 ≈ **2.86 Persons**
- This suggests a **core development team of 3 people**

6. Conclusion

The **JURMANA – Online Traffic Fine Payment System** has been analyzed and estimated using **Function Point Analysis** and the **Basic COCOMO model**. The system's design ensures comprehensive functionality, covering user authentication, fine issuance, payment integration, dispute management, notifications, and reporting.

From the **Function Point analysis**, the system size is estimated at **105 FP**, corresponding to approximately **6.3 KLOC** of code. Using the **COCOMO (Semi-Detached mode)** model with parameters (a = 3.0, b = 1.12, c = 0.35), the following results were derived:

- **Effort** ≈ 23.57 Person-Months
- **Development Time** ≈ 8.25 Months

- **Staff Requirement** ≈ 3 People
- **Total Development Cost (@₹50,000 per PM)** ≈ ₹11,78,500
- **Average Cost per Month** ≈ ₹1,42,800

These estimates suggest that the project is feasible with a **small, dedicated team** over a development timeline of about **8 months**. The combination of **Function Point sizing** and **COCOMO estimation** provides a reliable baseline for planning, resource allocation, scheduling, and budgeting.
The study confirms that **JURMANA is a cost-effective, efficient, and scalable solution** that can digitize traffic fine management, ensure transparency, and improve citizen convenience while supporting government traffic enforcement.

7. References

1) Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.
2) Sommerville, I. (2015). *Software Engineering*. Pearson Education.
3) Boehm, B. W. (1981). *Software Engineering Economics*. Prentice-Hall.